Université de Strasbourg

Master Informatique – 1ère année

Matière : Compilation

Compilateur pour le langage AFSTAT

Membres du groupe : Akofa Carole DAPAH
Line HAJJAR
Nicolas VENTADOUX

Enseignant : Philippe Clauss

18 janvier 2026

Table des matières

1 Structure du projet **2**
1.1 Dossier `include` 2
1.2 Dossier `source` 2
1.3 Dossier `tests` 2

2 Structures de données choisies **2**
2.1 AST (Arbre de Syntaxe Abstraite) 2
2.2 Table des symboles (`SYMBOL_TABLE`) 3

3 Analyse sémantique **3**

4 Génération de code (CODEGEN) **3**

5 Capacités du compilateur **4**
5.1 Fonctionnalités prises en charge 4
5.2 Colonnes calculées (`compute`) 4
5.3 Analyses statistiques supportées 4
5.4 Génération et exécution du code C 5
5.5 Options du compilateur 5

6 Jeux de tests et validation **5**

7 Points intéressants du projet **5**

8 Limites connues **6**

9 Conclusion **6**

1 Structure du projet

Le projet est organisé en plusieurs dossiers afin de séparer clairement les différentes parties du compilateur.

1.1 Dossier `include`

Il contient les fichiers d'en-tête suivants :

- `ast.h`
- `codegen.h`
- `semantic.h`
- `symbol_table.h`

1.2 Dossier `source`

Il contient les fichiers sources d'implémentation de notre compilateur AFSTAT :

- `ast.c`
- `codegen.c`
- `symbol_table.c`
- `semantic.c`

1.3 Dossier `tests`

Il contient les fichiers nécessaires à la validation du compilateur :

- fichiers AFSTAT (`.afs`) utilisés pour les tests,
- fichiers CSV associés,
- tests couvrant les cas simples et avancés (*analyse simple, compute, join, filter*).

2 Structures de données choisies

2.1 AST (Arbre de Syntaxe Abstraite)

L'AST permet de représenter la structure du programme AFSTAT après l'analyse syntaxique. La structure utilisée est un arbre dont chaque nœud contient :

- un type (énumérateur représentant la nature du nœud),
- un pointeur vers un texte (mot-clé, identifiant, opérateur),
- des pointeurs vers les nœuds enfants.

Les fonctionnalités principales sont :

- la construction de la racine,

— la construction des nœuds,
— l'affichage de l'arbre pour le débogage.

2.2 Table des symboles (`SYMBOL_TABLE`)

La table des symboles permet de stocker les informations issues du programme AFSTAT. Elle repose sur :

— une structure *colonne* (nom, type),
— une structure *schéma* (nom, source associée, liste de colonnes),
— une structure *table de symboles* contenant l'ensemble des schémas.

Cette organisation permet de vérifier la cohérence en faisant le lien entre les sources, les schémas et les fichiers CSV utilisés lors de l'exécution.

3 Analyse sémantique

La phase sémantique parcourt notre AST pour vérifier que notre programme est cohérent. Elle permet notamment de détecter :

— l'utilisation de sources non déclarées,
— des associations schéma/source incorrectes,
— des colonnes inexistantes,
— des incohérences de types,
— des analyses statistiques incompatibles,
— des erreurs liées aux jointures et aux filtres (colonnes inconnues/invalides, types incompatibles).

Les messages d'erreur produits sont explicites et permettent d'identifier précisément l'origine du problème avant la génération du code C, afin d'interrompre l'exécution.

4 Génération de code (CODEGEN)

La génération de code transforme un programme `.afs` en un programme C autonome compilable avec `gcc`. Le code C généré exécute réellement les opérations demandées sur les fichiers CSV.

Les principales fonctionnalités sont :

— génération de fonctions utilitaires pour la lecture et le découpage des fichiers CSV,
— traduction des opérations `analyze` en calculs statistiques en C (*mean, min, max, median, std_dev, summary, histogram*),

— implémentation de `join` et `filter` via la création de fichiers CSV temporaires utilisés comme *datasets* intermédiaires,
— exécution séquentielle du pipeline, dans l'ordre du programme AFSTAT.

5 Capacités du compilateur

Le compilateur AFSTAT permet de traduire un programme écrit dans le langage AFSTAT en un programme C correctement compilable et exécutable, produisant les résultats statistiques demandés.

5.1 Fonctionnalités prises en charge

Le compilateur supporte les constructions principales du langage AFSTAT :

— déclaration de sources de données CSV (`source`),
— définition de schémas (`schema`),
— association schéma/source (`associate`),
— colonnes calculées (`compute`),
— analyses statistiques (`analyze`),
— jointures entre sources (`join`),
— filtrage de données (`filter`).

Tous les programmes de test fournis dans l'énoncé ont été analysés, compilés et exécutés avec succès.

5.2 Colonnes calculées (`compute`)

Les colonnes définies par `compute` sont pleinement supportées :

— elles peuvent contenir des expressions arithmétiques,
— dépendre d'autres colonnes,
— être utilisées dans les analyses statistiques comme des colonnes classiques.

5.3 Analyses statistiques supportées

Les opérations statistiques suivantes sont implémentées :

— Mean,
— Maximum,
— Minimum,
— std_dev,
— summary (minimum, maximum, moyenne, écart-type),

— histogram.

Les résultats obtenus correspondent à ceux attendus dans l'énoncé, à de légères différences d'arrondi près dues aux calculs en virgule flottante.

5.4 Génération et exécution du code C

Le compilateur génère un fichier C autonome :

— compilable sans erreur avec `gcc -lm`,
— utilisant uniquement les bibliothèques standards,
— affichant directement les résultats à l'écran.

Le code généré est lisible, reproductible et exécutable sans modification manuelle.

5.5 Options du compilateur

Le compilateur propose plusieurs options :

— `-o` : spécifier le nom du fichier C généré,
— `-tos` : afficher la table des symboles,
— `-version` : afficher les membres du projet,
— `-tokens` : afficher les tokens,
— `-ast` : afficher l'AST.

6 Jeux de tests et validation

Le compilateur a été validé à l'aide des programmes AFSTAT fournis dans l'énoncé (`prog1` à `prog7`), couvrant :

— des analyses simples,
— des colonnes calculées,
— des jointures,
— des filtres,
— des analyses statistiques combinées.

Chaque programme a été traduit en C, compilé avec `gcc`, puis exécuté. Les résultats obtenus sont conformes aux résultats attendus.

7 Points intéressants du projet

Plusieurs aspects du compilateur sont particulièrement notables :

— la gestion unifiée des colonnes CSV et des colonnes calculées,

— la détection précoce des erreurs,
— la séparation claire des phases (syntaxique, sémantique, génération de code),
— la prise en charge de programmes AFSTAT relativement complexes.

8 Limites connues

Certaines limites ont été identifiées :

— Le parsing des fichiers CSV repose sur une séparation simple par virgule. Les champs entre guillemets contenant des virgules internes ne sont pas correctement interprétés et peuvent produire des résultats erronés.
— Seules les jointures de type *inner join* sont supportées ; les jointures *left*, *right* et *outer join* ne sont pas implémentées.
— Les conditions de filtrage sont limitées à des comparaisons simples sur une seule colonne ; les opérateurs logiques `AND` et `OR` ne sont pas pris en charge.

Ces limites sont connues, documentées et n'empêchent pas le compilateur de remplir les objectifs principaux du projet.

9 Conclusion

Le compilateur AFSTAT développé est fonctionnel, stable et conforme aux objectifs du projet. Il permet de transformer un langage métier orienté analyse statistique en un code C exécutable, tout en assurant une bonne détection des erreurs et des résultats corrects sur l'ensemble des tests fournis.

Plusieurs pistes d'amélioration peuvent être envisagées pour des versions futures, comme une optimisation du code généré ou l'ajout de fonctionnalités supplémentaires.